

Project_3_Solution

1 Training Data

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, recall_score, precision_score, \
    confusion_matrix
```

Loading training data. We split our training data into two parts. First 400 are used as training data, next 150 are used as validation data.

```
[2]: train_data = pd.read_csv('train_data.csv', index_col=0)
# Convert "Occupation" into a dummy variable
train_data.loc[train_data['OCCUPATION']!=1, 'OCCUPATION'] = 0
tr_data = train_data[0:400]
va_data = train_data[400:]
va_data.index = range(150)
```

```
[3]: train_data
```

```
[3]:
```

	DEBT	YRS_IN_RESIDENT	AGE	YRS_OF_EMPLOYMENT	DTI	NUM_PREV_APP	\
0	1600	3	19	1.50	2.000	0	
1	2200	1	48	4.25	0.125	0	
2	600	2	35	4.50	5.750	0	
3	200	4	20	0.50	0.000	3	
4	100	8	45	7.00	1.625	0	
..	...	...	...	...	...	...	
545	0	9	17	0.00	0.040	0	
546	1200	1	29	3.50	3.500	3	
547	100	5	27	14.50	3.085	1	
548	300	8	32	16.25	3.000	9	
549	600	1	32	3.00	3.040	2	

	OCCUPATION	PROVIDED_SIN	MARRIAGE	INCOME	EDUCATION	CREDIT_PROFILE	\
0	0	1	1	13	4	1	

1	1	1	2	7	4	1
2	1	1	2	14	7	0
3	0	1	2	1	1	0
4	0	1	1	8	4	0
..	...	...	...	...	...	...
545	0	1	1	8	4	0
546	1	1	2	9	4	1
547	0	0	2	14	7	1
548	0	1	2	2	4	1
549	0	0	2	1	1	0

APPROVAL_STATUS	
0	0
1	1
2	1
3	0
4	0
..	...
545	0
546	0
547	1
548	1
549	0

[550 rows x 13 columns]

Prepping data

```
[4]: # first, we list all the categorical variables to be one hot encoded
cat_vars = ['MARRIAGE', 'EDUCATION']
```

```
[5]: encoders = [OneHotEncoder(categories='auto') for _ in range(len(cat_vars))] #_
      ↪ create an encoder for each cat_vars
      # encode each of the cat_vars with their respective encoder
      encoded_tr = [encoders[i].fit_transform(tr_data[[cat_var]]).todense() for_
      ↪ i, cat_var in enumerate(cat_vars)]
      encoded_va = [encoders[i].transform(va_data[[cat_var]]).todense() for i, cat_var_
      ↪ in enumerate(cat_vars)]
```

```
[6]: # aggregating our data with the one hot encoded data
      # drop the label column and also drop the cat_vars
      # this way we can join the encoded categorical variables with the continuous_
      ↪ variables
      X_train = pd.concat([tr_data.iloc[:, :-1].drop(cat_vars, axis=1),
                          pd.DataFrame(np.concatenate(encoded_tr, axis=1))], axis=1)
      X_validation = pd.concat([va_data.iloc[:, :-1].drop(cat_vars, axis=1),
                              pd.DataFrame(np.concatenate(encoded_va, axis=1))], axis=1)
      y_train = tr_data.iloc[:, -1]
```

```

y_validation = va_data.iloc[:,-1]
X_train = X_train.rename(columns={0:'Marriage 1',1:'Marriage 2',2:'Marriage 3',3:'Edu 1',4:'Edu 2',5:'Edu 3',6:'Edu 4',7:'Edu 5',8:'Edu 6',9:'Edu 7'})
X_validation = X_validation.rename(columns={0:'Marriage 1',1:'Marriage 2',2:'Marriage 3',3:'Edu 1',4:'Edu 2',5:'Edu 3',6:'Edu 4',7:'Edu 5',8:'Edu 6',9:'Edu 7'})

```

```
[7]: X_train.head()
```

```

[7]:   DEBT  YRS_IN_RESIDENT  AGE  YRS_OF_EMPLOYMENT  DTI  NUM_PREV_APP  \
0   1600                3   19                1.50  2.000            0
1   2200                1   48                4.25  0.125            0
2    600                2   35                4.50  5.750            0
3    200                4   20                0.50  0.000            3
4    100                8   45                7.00  1.625            0

   OCCUPATION  PROVIDED_SIN  INCOME  CREDIT_PROFILE  Marriage 1  Marriage 2  \
0           0              1      13              1          1.0          0.0
1           1              1       7              1          0.0          1.0
2           1              1      14              0          0.0          1.0
3           0              1       1              0          0.0          1.0
4           0              1       8              0          1.0          0.0

   Marriage 3  Edu 1  Edu 2  Edu 3  Edu 4  Edu 5  Edu 6  Edu 7
0          0.0   0.0   0.0   0.0   1.0   0.0   0.0   0.0
1          0.0   0.0   0.0   0.0   1.0   0.0   0.0   0.0
2          0.0   0.0   0.0   0.0   0.0   0.0   0.0   1.0
3          0.0   1.0   0.0   0.0   0.0   0.0   0.0   0.0
4          0.0   0.0   0.0   0.0   1.0   0.0   0.0   0.0

```

```

[8]: # normalized continuous features
for i in [0,1,2,3,4,5,8]:
    X1 = X_train.iloc[:,i]
    mean = X1.mean()
    std = X1.std()
    X_train.iloc[:,i] = (X1-mean)/std
    X_validation.iloc[:,i] = (X_validation.iloc[:,i]-mean)/std

```

```
[9]: X_train.head()
```

```

[9]:   DEBT  YRS_IN_RESIDENT  AGE  YRS_OF_EMPLOYMENT  DTI  \
0  1.303140      -0.673039 -1.080653      -0.629013 -0.075700
1  1.993698      -1.358938  1.307986      -0.075482 -0.664258
2  0.152210      -1.015989  0.237217      -0.025161  1.101415
3 -0.308161      -0.330089 -0.998287      -0.830297 -0.703495
4 -0.423254       1.041710  1.060885       0.478050 -0.193412

```

	NUM_PREV_APP	OCCUPATION	PROVIDED_SIN	INCOME	CREDIT_PROFILE	\
0	-0.566394	0	1	1.540430	1	
1	-0.566394	1	1	-0.109294	1	
2	-0.566394	1	1	1.815384	0	
3	0.214839	0	1	-1.759018	0	
4	-0.566394	0	1	0.165660	0	

	Marriage 1	Marriage 2	Marriage 3	Edu 1	Edu 2	Edu 3	Edu 4	Edu 5	\
0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
1	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	
2	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	
3	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	
4	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	

	Edu 6	Edu 7
0	0.0	0.0
1	0.0	0.0
2	0.0	1.0
3	0.0	0.0
4	0.0	0.0

We use the regularized logistic regression to see results

```
[10]: lr = LogisticRegression(penalty='l1', solver='liblinear', C=1)
lr.fit(X_train, y_train)
```

```
[10]: LogisticRegression(C=1, penalty='l1', solver='liblinear')
```

```
[11]: print('Intercept = ', lr.intercept_[0])
#match coefs with categories
Ind=[i for i,x in enumerate(lr.coef_.flatten()) if x!=0]
dict(zip(X_train.columns[Ind], lr.coef_.flatten()[Ind]))
```

```
Intercept = -2.101152314226914
```

```
[11]: {'DEBT': 0.11768681882232587,
'YRS_IN_RESIDENT': 0.041815602752630264,
'AGE': -0.006527582136004946,
'YRS_OF_EMPLOYMENT': -0.11589534997500497,
'DTI': 0.16891533201159875,
'NUM_PREV_APP': 0.655205177092516,
'PROVIDED_SIN': -0.18426777823770057,
'INCOME': 0.5739896956683423,
'CREDIT_PROFILE': 3.3782030300511234,
'Marriage 1': -0.21267328324949555,
'Edu 1': -0.017575980503405156,
'Edu 4': -0.06054979324564452,
```

```
'Edu 5': -0.42804619889822576,
'Edu 6': 1.24998687421493,
'Edu 7': 0.16922147574462454}
```

```
[12]: n = X_validation.shape[0]
pred = lr.predict(X_validation)
TN,FP,FN,TP = confusion_matrix(y_validation, pred).ravel()
print('Accuracy: ', accuracy_score(y_validation, pred))
print('Precision: ', precision_score(y_validation, pred))
print('Recall (True Positive): ', recall_score(y_validation, pred))
print('True Negative Rate: ', TN/(TN+FP))
print('Sum of True Positive Rate and True Negative Rate', TP/(TP+FN)+TN/(TN+FP))
```

```
Accuracy: 0.8466666666666667
Precision: 0.7948717948717948
Recall (True Positive): 0.8985507246376812
True Negative Rate: 0.8024691358024691
Sum of True Positive Rate and True Negative Rate 1.7010198604401503
```

```
[13]: # Try different thresholds, but there is not a lot of improvement because the
      ↪ dataset is rather balanced
Q = lr.predict_proba(X_validation)[: ,1]
THRESHOLD = [0.4, 0.45, 0.5, 0.55, 0.6]
for i in THRESHOLD:
    print('\nTHRESHOLD = ',i)
    pred = np.where(Q>i,1,0)
    TN,FP,FN,TP = confusion_matrix(y_validation, pred).ravel()
    print('Accuracy: ', accuracy_score(y_validation, pred))
    print('Precision: ', precision_score(y_validation, pred))
    print('Recall (True Positive): ', recall_score(y_validation, pred))
    print('True Negative Rate: ', TN/(TN+FP))
    print('Sum of True Positive Rate and True Negative Rate', TP/(TP+FN)+TN/
      ↪ (TN+FP))
```

```
THRESHOLD = 0.4
Accuracy: 0.8333333333333334
Precision: 0.775
Recall (True Positive): 0.8985507246376812
True Negative Rate: 0.7777777777777778
Sum of True Positive Rate and True Negative Rate 1.6763285024154588
```

```
THRESHOLD = 0.45
Accuracy: 0.84
Precision: 0.7848101265822784
Recall (True Positive): 0.8985507246376812
True Negative Rate: 0.7901234567901234
Sum of True Positive Rate and True Negative Rate 1.6886741814278046
```

```

THRESHOLD = 0.5
Accuracy: 0.8466666666666667
Precision: 0.7948717948717948
Recall (True Positive): 0.8985507246376812
True Negative Rate: 0.8024691358024691
Sum of True Positive Rate and True Negative Rate 1.7010198604401503

```

```

THRESHOLD = 0.55
Accuracy: 0.8466666666666667
Precision: 0.7948717948717948
Recall (True Positive): 0.8985507246376812
True Negative Rate: 0.8024691358024691
Sum of True Positive Rate and True Negative Rate 1.7010198604401503

```

```

THRESHOLD = 0.6
Accuracy: 0.84
Precision: 0.8
Recall (True Positive): 0.8695652173913043
True Negative Rate: 0.8148148148148148
Sum of True Positive Rate and True Negative Rate 1.684380032206119

```

```

[14]: from sklearn.model_selection import GridSearchCV # this is a way to search for
      ↪ best parameters

```

```

[15]: # Try L2 regularization instead of L1 regularization
lr = LogisticRegression(penalty='l2', solver='liblinear') # l2 regularization
      ↪ parameter
params = {'C': [i/100 for i in range(1,101)]} # creates a list of possible
      ↪ parameters we want to try

```

The following is our grid search function, searching through all parameters in param. It looks for the parameters that maximizes accuracy, precision, and recall. Since there are three things to improve on, we pick recall as the one that determines the best parameter. (This is an arbitrary choice)

```

[16]: clf = GridSearchCV(lr, params, cv=5, scoring='recall')

```

```

[17]: clf.fit(X_train, y_train)

```

```

[17]: GridSearchCV(cv=5, estimator=LogisticRegression(solver='liblinear'),
      param_grid={'C': [0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08,
                        0.09, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, 0.16,
                        0.17, 0.18, 0.19, 0.2, 0.21, 0.22, 0.23, 0.24,
                        0.25, 0.26, 0.27, 0.28, 0.29, 0.3, ...]},
      scoring='recall')

```

```

[18]: clf.best_params_ # the best parameter

```

```
[18]: {'C': 0.62}
```

```
[19]: lr = LogisticRegression(penalty='l2',solver='liblinear',C=0.8)
lr.fit(X_train, y_train)
pred = lr.predict(X_validation)
TN,FP,FN,TP = confusion_matrix(y_validation, pred).ravel()
print('Accuracy: ', accuracy_score(y_validation, pred))
print('Precision: ', precision_score(y_validation, pred))
print('Recall (True Positive): ', recall_score(y_validation, pred))
print('True Negative Rate: ',TN/(TN+FP))
print('Sum of True Positive Rate and True Negative Rate',TP/(TP+FN)+TN/(TN+FP))
```

```
Accuracy:  0.8533333333333334
Precision:  0.7974683544303798
Recall (True Positive):  0.9130434782608695
True Negative Rate:  0.8024691358024691
Sum of True Positive Rate and True Negative Rate 1.7155126140633388
```

```
[20]: # Save and output the model for future usage
import pickle
filename = 'finalized_model.sav'
pickle.dump(lr, open(filename, 'wb'))
```

1.1 This is to test the model out-of-sample

```
[21]: # delete all variables and functions in namespace
%reset
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, recall_score, precision_score,
↪confusion_matrix
```

Once deleted, variables cannot be recovered. Proceed (y/[n])? y

```
[22]: test_data = pd.read_csv('test_data.csv',index_col=0)
train_data=pd.read_csv('train_data.csv',index_col=0)
# Convert "Occupation" into a dummy variable
test_data.loc[test_data['OCCUPATION']!=1,'OCCUPATION'] = 0
train_data.loc[train_data['OCCUPATION']!=1,'OCCUPATION'] = 0
total_data=pd.concat([train_data,test_data])
total_data=total_data.reset_index(drop=True)
```

```
[23]: # first, we list all the categorical variables to be one hot encoded
cat_vars = ['MARRIAGE', 'EDUCATION']
```

```
[24]: encoders = [OneHotEncoder(categories='auto') for _ in range(len(cat_vars))] #  

    ↪ create an encoder for each cat_vars  

    # encode each of the cat_vars with their respective encoder  

    encoded_total = [encoders[i].fit_transform(total_data[[cat_var]]).todense() for  

    ↪ i, cat_var in enumerate(cat_vars)]
```

```
[25]: # aggregating our data with the one hot encoded data  

    # drop the label column and also drop the cat_vars  

    # this way we can join the encoded categorical variables with the continuous  

    ↪ variables  

    X_total = pd.concat([total_data.iloc[:, :-1].drop(cat_vars, axis=1),  

        pd.DataFrame(np.concatenate(encoded_total, axis=1))],  

    ↪ axis=1)  

  

    y_test = test_data.iloc[:, -1]  

    X_test=X_total.tail(len(y_test))  

    X_test= X_test.rename(columns={0: 'Marriage 1', 1: 'Marriage 2', 2: 'Marriage 3', 3:  

    ↪ 'Edu 1', 4: 'Edu 2', 5: 'Edu 3',  

        6: 'Edu 4', 7: 'Edu 5', 8: 'Edu 6', 9: 'Edu 7'})  

  

    X_train=X_total[0:len(X_total)-len(y_test)]
```

```
[26]: # normalized continuous features  

    for i in [0,1,2,3,4,5,8]:  

        X1 = X_train.iloc[:, i]  

        mean = X1.mean()  

        std = X1.std()  

        X_test.iloc[:, i] = (X_test.iloc[:, i] - mean) / std
```

```
[27]: X_test.head()
```

```
[27]:
```

	DEBT	YRS_IN_RESIDENT	AGE	YRS_OF_EMPLOYMENT	DTI	\
550	-0.157078	-1.025796	1.645828	-0.543313	-0.222781	
551	-0.386238	-0.008630	-1.334272	-0.394101	-0.470181	
552	-0.386238	-0.008630	-0.920369	-0.891475	-0.593140	
553	-0.004305	1.008535	1.149145	1.147759	0.814224	
554	-0.386238	-0.686741	-0.175344	-0.543313	-0.655360	

	NUM_PREV_APP	OCCUPATION	PROVIDED_SIN	INCOME	CREDIT_PROFILE	\
550	-0.555121	0	1	-1.749370	0	
551	-0.318808	0	1	-0.382675	0	
552	-0.555121	0	1	0.164003	0	
553	1.099071	1	0	0.984020	1	
554	-0.555121	0	1	1.804037	0	

	Marriage 1	Marriage 2	Marriage 3	Edu 1	Edu 2	Edu 3	Edu 4	Edu 5	\
550	1.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	

551	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0
552	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
553	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0
554	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

	Edu 6	Edu 7
550	0.0	0.0
551	0.0	0.0
552	0.0	1.0
553	0.0	0.0
554	0.0	1.0

```
[28]: # Import the saved model from the training data
import pickle
lr_load = pickle.load(open('finalized_model.sav', 'rb'))
```

```
[29]: # Evaluate the out-of-sample performance
pred = lr_load.predict(X_test)
TN,FP,FN,TP = confusion_matrix(y_test, pred).ravel()
print('Accuracy: ', accuracy_score(y_test, pred))
print('Precision: ', precision_score(y_test, pred))
print('Recall (True Positive): ', recall_score(y_test, pred))
print('True Negative Rate: ', TN/(TN+FP))
print('Sum of True Positive Rate and True Negative Rate', TP/(TP+FN)+TN/(TN+FP))
```

```
Accuracy: 0.90625
Precision: 0.8571428571428571
Recall (True Positive): 0.9473684210526315
True Negative Rate: 0.8732394366197183
Sum of True Positive Rate and True Negative Rate 1.82060785767235
```